Assignment 01 - Report

Customer Support Ticket Dashboard

1. Project Overview

The project is a browser-based Customer Support Ticket Dashboard for a SaaS company. It is built with HTML, CSS, Bootstrap 5 and JavaScript (ES6 modules) and needs no backend. Support agents can create tickets, edit and delete them, search by issue keyword, filter by status and priority, and see live totals of open, in-progress and closed tickets.

The code is split into three modules: ticketManager.js (a TicketManager class holding the data and operations), validation.js (input rules) and main.js (DOM wiring and rendering). Responsive behaviour comes from Bootstrap's grid together with a CSS Grid page layout that stacks on mobile and becomes two columns on desktop.

2. Implementation

This section shows the working dashboard and explains how each feature of Part A and Part B behaves. The code is split into three ES6 modules: ticketManager.js holds the data and operations, validation.js holds the input rules, and main.js connects them to the page.

2.1 Part A: Core Features

Responsive layout and ticket view. Figure 1 shows the dashboard on a desktop screen. The four summary cards use Bootstrap's grid classes, so they sit in one row on large screens and wrap into two rows on phones. The form and the ticket list sit in a CSS Grid container that has two columns on desktop and a single column on mobile. The top bar and the form panel use position: sticky so they stay in view while the page scrolls. The ticket table is built by a JavaScript function that loops over the ticket array and writes one row per ticket, together with the priority and status badges.

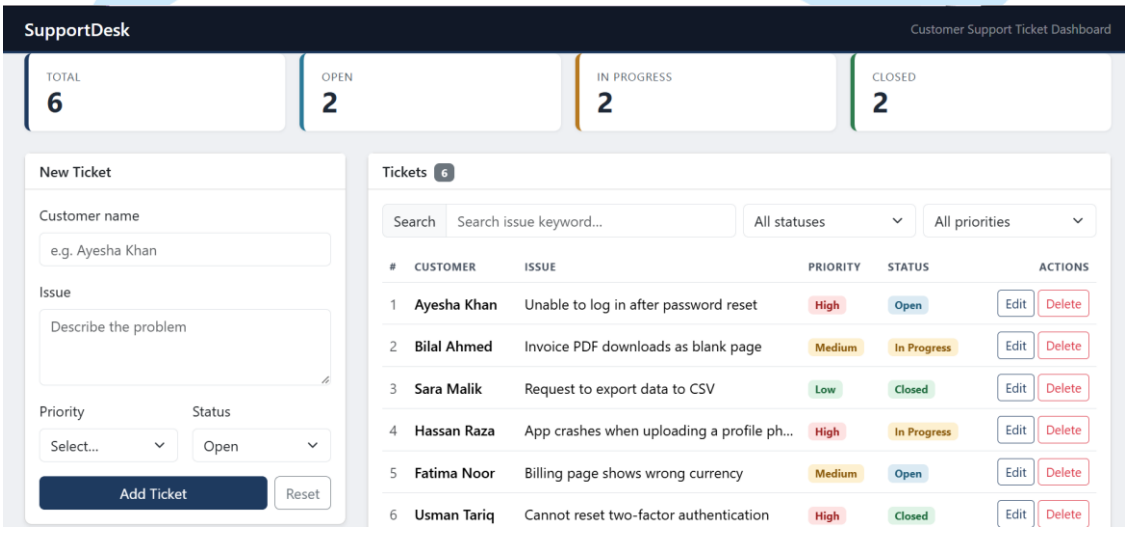


Figure 1: Dashboard on a desktop screen showing the summary cards, ticket form and ticket list

Form validation. In Figure 2 the form was submitted with an empty name, a very short issue and no priority. The validateTicket function checks each field with conditions and comparison operators: the name must not be empty, must be 2 to 50 characters and may contain only letters, spaces and a few punctuation marks. The issue must be at least 10 and at most 300 characters. The priority and status must be one of the allowed values. The function returns an object of error messages, and the page marks each invalid field in red and prints its message below it. The ticket is not saved until every error is cleared.

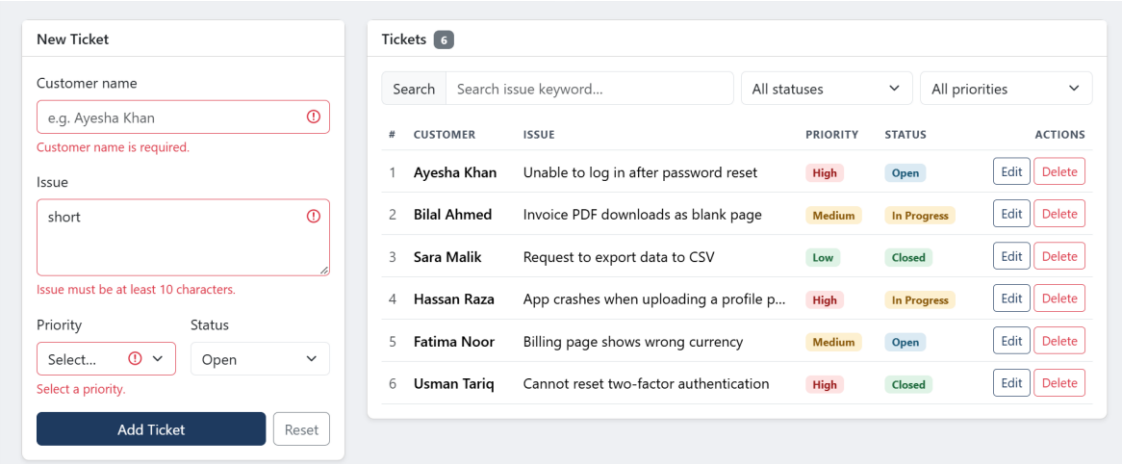


Figure 2: Validation messages shown when the form is submitted with invalid input

Adding a ticket and live counts. Figure 3 shows the dashboard after a valid ticket was submitted. The submit handler reads the field values, validates them, and then calls the add method of the TicketManager class. The method creates a ticket object with a new id and the trimmed values and adds it to the array using the spread operator. The page then renders the table and the summary again, which is why the total changed from 6 to 7 and the Open count from 2 to 3 without reloading the page. The Reset button calls a function that clears the fields, restores the default status of Open, removes any error messages and returns the form from edit mode to add mode, also without a page reload.

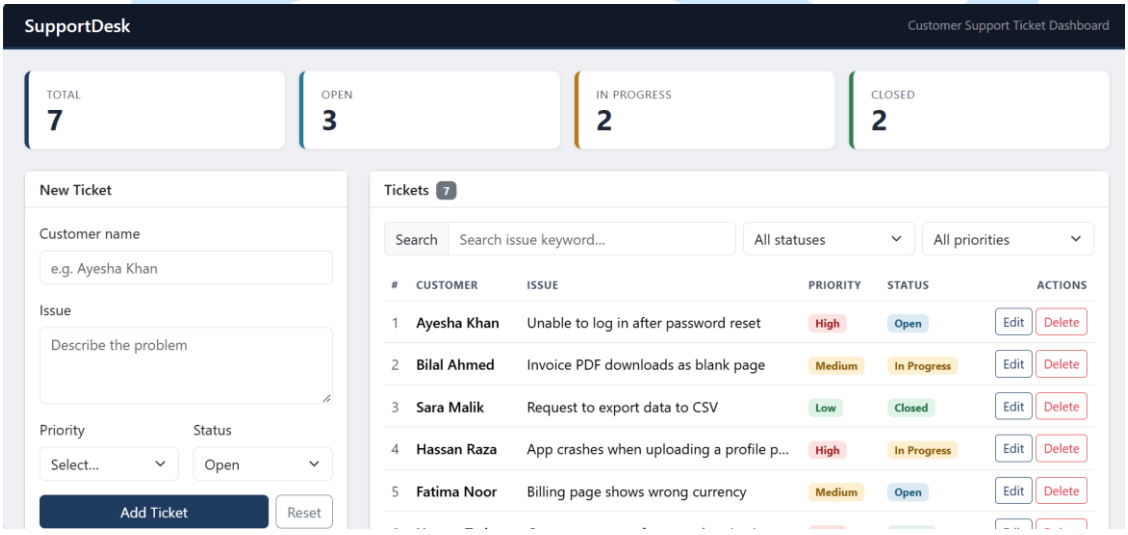


Figure 3: Dashboard after a new ticket was added, with the totals updated

2.2 Part B: Additional Features

Editing a ticket. Each row has an Edit button. In Figure 4 (shown at a narrow screen width, where the cards and form stack vertically) the button for ticket 2 was pressed. The startEdit function finds the ticket in the array, destructures its fields, copies them into the form, changes the heading to Edit Ticket #2 and changes the button label to Save Changes. When the form is submitted, the update method goes through the array with map and replaces only the matching ticket with a new object built using the spread operator, so the other tickets are left untouched.

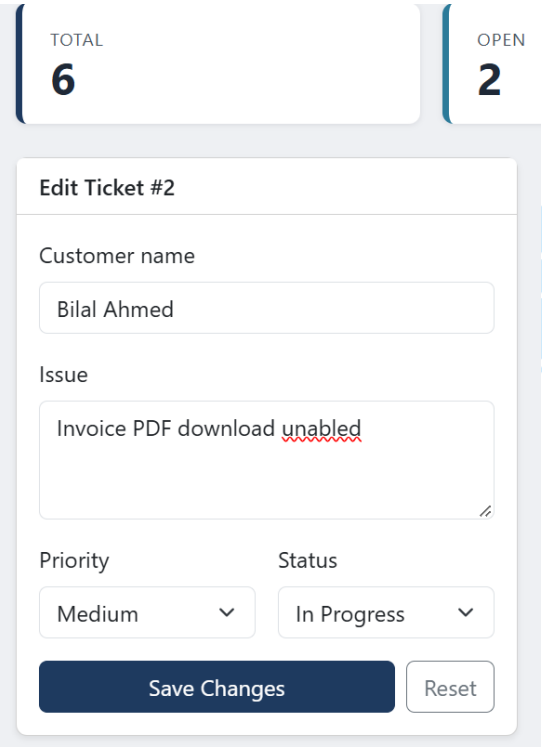


Figure 4: Form in edit mode for ticket 2

Search and filter. In Figure 5 the word log was typed in the search box and the priority filter was set to High. The search method of the TicketManager class uses the array filter method with three conditions joined by the logical AND operator: the issue must contain the keyword (compared in lower case), the status must match the selected status if one is chosen, and the priority must match the selected priority if one is chosen. Only the matching ticket remains in the table, the matched text is highlighted, and the badge next to the heading shows the number of visible tickets. The summary cards continue to show totals for all tickets, not just the filtered ones.

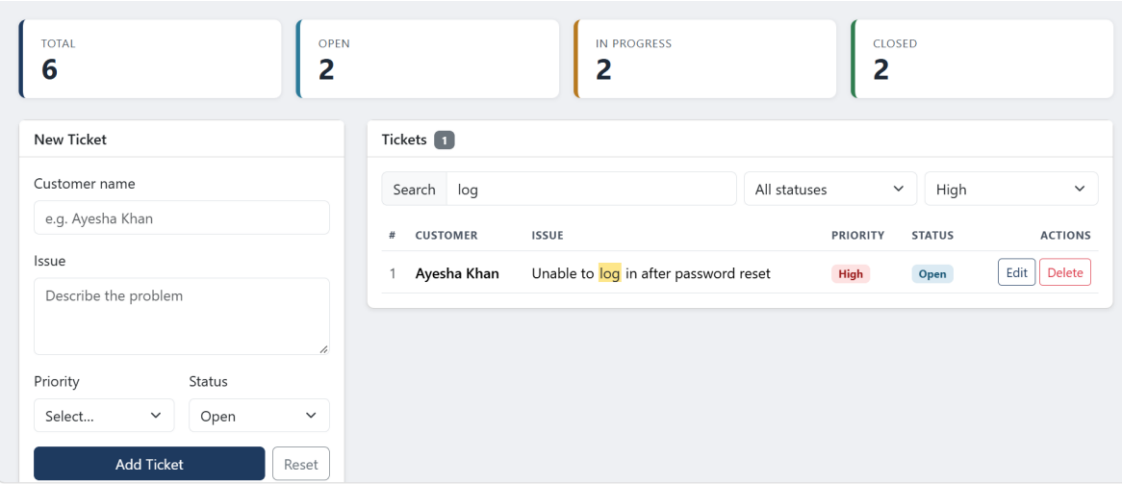


Figure 5: Search keyword and priority filter applied, with the match highlighted

Deleting a ticket and the summary totals. Pressing Delete asks for confirmation and then calls the remove method, which keeps every ticket except the selected one by using the array filter method. Figure 6 shows the dashboard after tickets were deleted. The summary method then loops over the remaining tickets, destructures each status and increments the matching counter, so the cards show a total of 3 with 1 open, 1 in progress and 1 closed.

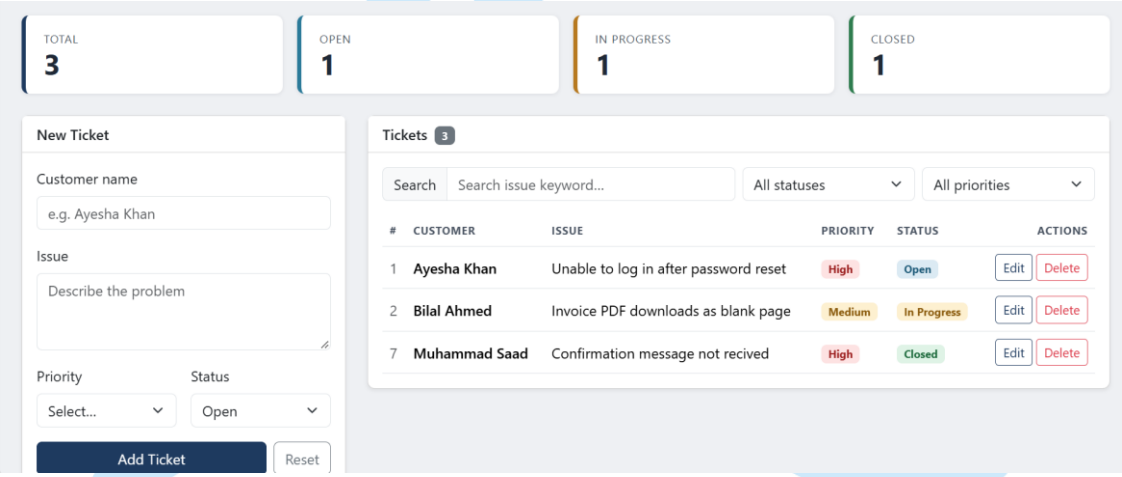


Figure 6: Dashboard after deleting tickets, with the summary recalculated

ES6 code structure. The required ES6 features are used as follows: arrow functions for the helpers and event handlers in main.js, destructuring when reading ticket fields and form values, the spread operator when adding and updating tickets, the TicketManager class with a private ticket list, and import and export statements to share code between the three modules.

2.3 Key Code

The snippets below are taken directly from the project source and show where the main requirements of Parts A and B are implemented.

TicketManager class with add and update (ticketManager.js)

JS ticketManager.js

```
export class TicketManager {
  #tickets = [];
  #nextId = 1;

  // ...

  // CREATE: build an object, spread it into the array
  add({ customer, issue, priority, status }) {
    const ticket = {
      id: this.#nextId++,
      customer: customer.trim(),
      issue: issue.trim(),
      priority,
      status,
      createdAt: new Date().toISOString(),
    };
    this.#tickets = [...this.#tickets, ticket];
    return ticket;
  }

  // UPDATE: map + spread to produce a new object without mutating the old one
  update(id, changes) {
    let updated = null;
    this.#tickets = this.#tickets.map((t) => {
      if (t.id !== id) return t;
      updated = { ...t, ...changes };
      return updated;
    });
    return updated;
  }
}
```

The class keeps the tickets in a private array. The add method builds a ticket object and appends it with the spread operator, and the update method uses map and spread to return a new array in which only the matching ticket is replaced.

Search and filter (ticketManager.js)

JS ticketManager.js

```
// SEARCH & FILTER: keyword in issue, plus status and priority
search({ keyword = "", status = "", priority = "" } = {}) {
  const term = keyword.trim().toLowerCase();
  return this.#tickets.filter(
    (t) =>
      (term === "" || t.issue.toLowerCase().includes(term)) &&
      (status === "" || t.status === status) &&
      (priority === "" || t.priority === priority)
  );
}
```

The filter method keeps a ticket only when the keyword, the status and the priority conditions are all true. An empty value means that condition is not applied.

Ticket summary (ticketManager.js)

JS ticketManager.js

```
// SUMMARY: totals computed with a loop
summary() {
  const counts = { total: 0, open: 0, progress: 0, closed: 0 };
  for (const { status } of this.#tickets) {
    counts.total++;
    if (status === "Open") counts.open++;
    else if (status === "In Progress") counts.progress++;
    else if (status === "Closed") counts.closed++;
  }
  return counts;
}
```

A for...of loop with destructuring reads the status of each ticket and increments the matching counter, and the totals object is returned to the page.

Input validation (validation.js)

JS validation.js

```
export const validateTicket = ({ customer, issue, priority, status }) => {
  const errors = {};
  const name = customer.trim();
  const text = issue.trim();

  if (name === "") {
    errors.customer = "Customer name is required.";
  } else if (name.length < 2 || name.length > 50) {
    errors.customer = "Name must be between 2 and 50 characters.";
  } else if (!NAME_PATTERN.test(name)) {
    errors.customer = "Name may contain only letters, spaces and . ' -";
  }

  if (text === "") {
    errors.issue = "Please describe the issue.";
  } else if (text.length < 10) {
    errors.issue = "Issue must be at least 10 characters.";
  } else if (text.length > 300) {
    errors.issue = "Issue must be 300 characters or fewer.";
  }

  if (!PRIORITIES.includes(priority)) {
    errors.priority = "Select a priority.";
  }

  if (!STATUSES.includes(status)) {
    errors.status = "Select a valid status.";
  }

  return errors;
};
```

Each field is checked with conditions and comparison operators, and the messages are collected in an errors object. An empty object means the input is valid.

ES6 modules

```
JS  import / export

1   // main.js
2   import { TicketManager } from "./ticketManager.js";
3   import { validateTicket } from "./validation.js";
4
5   // validation.js
6   import { STATUSES, PRIORITIES } from "./ticketManager.js";
7   export const validateTicket = ({ customer, issue, priority, status }) => {
8     // ...
9   };
10
11  // ticketManager.js
12  export const STATUSES = ["Open", "In Progress", "Closed"];
13  export const PRIORITIES = ["Low", "Medium", "High"];
14  export class TicketManager {
15    // ...
16  }
```

The code is divided into three files. Each file exports what the others need and imports it with a path, which keeps the data, validation and page logic separate.

Rendering the ticket rows (main.js)

```
JS  main.js

1   els.body.innerHTML = rows
2     .map(({ id, customer, issue, priority, status }) => `
3       <tr>
4         <td class="text-muted">${id}</td>
5         <td class="fw-semibold">${escapeHtml(customer)}</td>
6         ...
7       </tr>`)
8     .join("");
```

An arrow function with a destructured parameter is passed to map, which turns every ticket into a row of HTML. Some cells are left out here to keep the snippet short.

Responsive layout with CSS Grid (styles.css)

```
#  styles.css

1   .dashboard-grid {
2     display: grid;
3     grid-template-columns: 1fr;
4     gap: 1.5rem;
5   }
6   @media (min-width: 992px) {
7     .dashboard-grid {
8       grid-template-columns: 360px 1fr;
9       align-items: start;
10    }
11    /* sticky keeps the form in view while the list scrolls */
12    .form-panel {
13      position: sticky;
14      top: 5rem;
15    }
16  }
```

The page is a single column by default. From 992px upwards the grid becomes two columns, with a fixed form column and a flexible ticket list, and the form panel stays in view while scrolling.

3. How Agile/Scrum Was Followed

Scrum was adapted to a one-person team over the one-week window (17-24 October 2026). The roles, events and artifacts were kept, but at a smaller scale, which is a common and accepted way to apply Scrum to individual coursework.

3.1 Scrum Roles

Role	How it was filled
Product Owner	The assignment brief acted as the stakeholder voice. The developer prioritised the backlog against its requirements (Parts A and B) and the grading outcomes (CLO-1 to CLO-3).
Scrum Master	The developer kept the process on track: protecting sprint scope, removing blockers (for example, deciding to use a local web server so ES modules load) and keeping the Definition of Done.
Development Team	The developer designed, coded and tested every increment.

3.2 Product Backlog

User stories were written from the brief and ordered by value. Story points use a simple 1-5 relative scale.

ID	User story	Priority	Points	Sprint
PB-1	As an agent, I want a responsive layout so I can use the dashboard on phone and desktop.	High	3	Sprint 1
PB-2	As an agent, I want a ticket form (name, issue, priority, status) so I can record customer issues.	High	3	Sprint 1
PB-3	As an agent, I want invalid input rejected with clear messages so the data stays clean.	High	2	Sprint 1
PB-4	As an agent, I want to see all tickets and counts so I know the workload.	High	2	Sprint 1
PB-5	As an agent, I want to reset the form without reloading the page.	Medium	1	Sprint 1
PB-6	As an agent, I want to edit and delete tickets so records stay correct.	High	3	Sprint 2
PB-7	As an agent, I want to search by keyword and filter by status/priority to find tickets quickly.	High	3	Sprint 2
PB-8	As a developer, I want ES6 structure (arrow functions, destructuring, spread, class, modules) for maintainability.	Medium	3	Sprint 2
PB-9	As a manager, I want totals for open, in-progress and closed tickets.	Medium	2	Sprint 2

ID	User story	Priority	Points	Sprint
PB-10	As a student, I want a report comparing Agile/Scrum and RUP.	High	3	Sprint 3

3.3 Sprint Planning

Three short sprints were planned. Each began with a planning step that selected backlog items, set a sprint goal and defined what would be demonstrable at the end.

Sprint	Dates	Sprint goal	Increment delivered
Sprint 1	17-19 Oct	A usable, responsive page that can capture and display tickets.	Bootstrap layout, validated form, ticket table with counts, working Reset.
Sprint 2	20-22 Oct	Complete ticket management and ES6 code structure.	Edit/delete, search and filters, summary totals, refactor into modules and a TicketManager class.
Sprint 3	23-24 Oct	Finish, polish and document.	Responsive polish, testing, this report, final packaging for submission.

3.4 Daily Standups

A short self-standup was held at the start of each working day, answering three questions: what was finished yesterday, what is planned today, and what is blocking progress. A typical entry:

- **Yesterday:** form fields and layout finished. **Today:** write validation rules and render the ticket table. **Blockers:** ES modules do not load from file://, so a local server is needed.

3.5 Other Scrum Events

- **Sprint Review:** at the end of each sprint the working page was opened in the browser and checked against the backlog items, including a phone-width view.
- **Sprint Retrospective:** after each sprint, a few lines on what worked and what to change. Sprint 1 showed that validation logic was getting mixed into the UI code, so Sprint 2 started by extracting it into validation.js.
- **Definition of Done:** story meets its acceptance criteria, has no console errors, works at mobile and desktop widths, and the logic passes a quick test of add, edit, delete, search and summary.

3.6 Increments

Each sprint ended with a working, shippable increment that built on the last: Increment 1 (layout, form, table), Increment 2 (full ticket management, search, filters, summary, ES6 structure) and Increment 3 (polished product plus documentation).

4. Rational Unified Process (RUP) Plan

RUP is an iterative, use-case-driven process organised into four phases, each ending with a milestone. The following plan applies it to the same dashboard project.

Phase	Objectives and activities	Deliverables / milestone
Inception	Establish the business case and scope. Identify stakeholders (support agents, support manager). Capture the main use cases: create ticket, edit ticket, delete ticket, search/filter, view summary. Identify top risks (unclear requirements, browser compatibility) and decide feasibility.	Vision document, initial use-case model, risk list, rough schedule. Milestone: Lifecycle Objectives.
Elaboration	Detail the requirements and design a stable architecture. Define the data model (Ticket object: id, customer, issue, priority, status). Design the module structure and UI wireframes. Build an architectural prototype (form plus table) to retire technical risks.	Use-case specifications, class and sequence diagrams, wireframes, executable architecture baseline. Milestone: Lifecycle Architecture.
Construction	Build the remaining features over iterations: validation, edit/delete, search/filter, summary, ES6 refactor. Unit-test each iteration and integrate continuously.	Complete, tested dashboard (beta release), user guide draft. Milestone: Initial Operational Capability.
Transition	Prepare for hand-over. Fix defects found in acceptance testing, test on different browsers and screen sizes, finalise documentation and package the files for delivery.	Final release, report, source package. Milestone: Product Release.

5. Comparison: Agile/Scrum vs RUP

Aspect	Agile / Scrum	RUP
Iteration	Fixed, short time-boxed sprints (here 2-3 days). Each sprint repeats the plan-build-review cycle.	Iterations are grouped inside four phases. Iteration length is flexible and early iterations focus on analysis and architecture.
Planning	Lightweight and continuous. The product backlog is reprioritised and sprint planning commits only to the next small slice.	More upfront planning. A phase plan and overall project plan exist from Inception, and are refined at each milestone.
Feedback	Frequent and fast: daily standups, sprint review and retrospective give feedback at least every sprint.	Feedback comes at each iteration and is formally assessed at phase milestones, so the loop is usually longer.
Deliverables	A working, potentially shippable increment each sprint. Documentation is kept minimal.	A defined set of artifacts (vision, use-case models, design models, test plans) as well as the working software.

Aspect	Agile / Scrum	RUP
Roles	Three roles: Product Owner, Scrum Master, Development Team.	Many specialised roles (analyst, architect, developer, tester, project manager).
Change handling	Change is welcome at any sprint boundary through backlog reprioritisation.	Change is managed, with most major requirement decisions settled by the end of Elaboration.
Risk	Reduced by short cycles and early working software.	Risk-driven: highest risks are tackled first, mainly in Elaboration.
Best suited to	Small teams, evolving requirements, fast delivery.	Larger, more complex systems that need strong architecture and traceable documentation.

5.1 Which Fits This Assignment Better?

For this dashboard, Agile/Scrum is the better fit. The scope is small, the team is one person, the deadline is one week and the requirements are clear but may be refined while building. Short sprints with a working increment each time gave quick feedback and kept the work on schedule. RUP's heavier documentation and formal phase milestones would add overhead that this size of project does not need. However, RUP's emphasis on early risk reduction and architecture was still useful as a way of thinking: deciding the module structure and data model before writing the feature code is exactly what the Elaboration phase prescribes.

AIR UNIVERSITY